Vault with AD

Vault

Vault is an identity-based secrets and encryption management system. A secret is anything that you want to tightly control access to, such as API encryption keys, passwords, or certificates. Vault provides encryption services that are gated by authentication and authorization methods. Using Vault's UI, CLI, or HTTP API, access to secrets and other sensitive data can be securely stored and managed, tightly controlled (restricted), and auditable.

Hashicorp Vault documentation

First setup for back-end

Copy `vault.hcl` to `/etc/vault.d/` and start the back-end:

- Manually...

Bash

```bash
sudo nohup vault server -config=/etc/vault.d/vault.hcl > /var/log/vault-manual.log &
```

- ...or as a systemd service

Bash

```bash
sudo systemctl enable vault.service && sudo systemctl start vault.service
```

Then initialize the host and NOTE DOWN THE GENERATED KEYS:

Bash

```bash
vault operator init
```

Vault starts in a sealed state, to unseal, use the following command three times:

```bash
vault operator unseal
```

Back-end should now be running and Vault should be unsealed.

Write policies

Create `hpc-default` policy. Applied to all logins with AD group `HPC Centre`:

```bash
vault policy write hpc-default hpc-default.hcl
```

Azure AD authentication method setup

NOTE: users with over 200 groups might run into problems. Additional setup needed to accommodate users with over 200 groups. Check the relevant chapter in Vault official docs HERE!

Authenticate as root with the generated root token:

```bash
vault login
```

Enable OIDC auth:

```bash
vault auth enable oidc
```

Configure OIDC for Azure AD with the default role:

```bash
```

```
AUTH0_DOMAIN="login.microsoftonline.com/<YOUR-TENANT-ID-HERE>/v2.0"
AUTH0_CLIENT_ID="<YOUR-AZURE-APP-ID-HERE>"
AUTH0_CLIENT_SECRET="<YOUR-AZURE-APP-SECRET-HERE>"
AUTH0_DEFAULT_ROLE="aad"

vault write auth/oidc/config \
   oidc_discovery_url="https://$AUTH0_DOMAIN" \
   oidc_client_id="$AUTH0_CLIENT_ID" \
   oidc_client_secret="$AUTH0_CLIENT_SECRET" \
   default_role="$AUTH0_DEFAULT_ROLE"
```

Configure the default role

Bash

```
AUTH0_USER_CLAIM="email"
AUTH0_ALLOWED_REDIRECT_URI="http://localhost:8250/oidc/callback,$VAULT_ADDR/ui
AUTH0_HPC_POLICY="default"
AUTH0_TTL="1h"
AUTH0_OIDC_SCOPES="profile"
HPC_AZURE_GROUP_ID="<YOUR-AZURE-APP-ID-HERE>"


vault write auth/oidc/role/aad -<<EOF
{
  "user_claim": "$AUTH0_USER_CLAIM",
  "allowed_redirect_uris": "$AUTH0_ALLOWED_REDIRECT_URI",
  "policies": "$AUTH0_HPC_POLICY",
  "ttl": "$AUTH0_TTL",
  "oidc_scopes": "$AUTH0_OIDC_SCOPES",
  "bound_claims": { "groups": ["$HPC_AZURE_GROUP_ID"] }
}
EOF
```

Link default role and policy for users with AD group `HPC Centre`:

Bash

```
```

```
AUTH0_ACCESSOR=$(vault auth list -format=json | jq -r '."oidc/".accessor')
HPC_VAULT_GROUP_NAME="HPC Centre"
HPC_VAULT_GROUP_POLICY="hpc-default"
HPC_VAULT_GROUP_ID=$(vault write \
   -field=id -format=table \
   identity/group \
   name="$HPC_VAULT_GROUP_NAME" \
   type="external" \
   policies="$HPC_VAULT_GROUP_POLICY")

vault write identity/group-alias \
   name="$HPC_AZURE_GROUP_ID" \
   mount_accessor="$AUTH0_ACCESSOR" \
   canonical_id="$HPC_VAULT_GROUP_ID"
```

Key/Value version 1 engine setup

The `kv` secrets engine is used to store arbitrary secrets within the configured physical storage for Vault. Writing to a key in the `kv` backend will replace the old value; sub-fields are not merged together.

Enable a version 1 kv for personal secrets:

Bash

```bash
vault secrets enable -path="personal" kv
```

According to `hpc-default` policy:

- Every `HPC Centre` group user can store personal secrets that only they can modify and read at the templated path `personal/{{identity.entity.aliases.$AUTH0_ACCESSOR.name}}/*`
- Every `HPC Centre` group user can create write-only secrets to any other users path `personal/*` (Existing values can not be modified/overwritten).

Enable a version 1 kv for general secrets:

Bash

```bash
vault secrets enable -path="secret" kv
```

General secrets should follow a defined structure that is agreed upon beforehand. The more granular the paths are, the easier it will be to manage these secrets as the amount of secrets grows. In essence paths should be something like the following example:

Bash

```bash
secret/
├── hpc-default/
│   └── base/
│        └── testsecret="Sup3rS3cr3t"
│
├── hpc-sysadmin/
│   └── clusters/
│        ├── amp/
│        ├── grey/
│        └── green/
│
└── hpc-dev/
    └── invenio/
```

Userpass authentication method setup

The `userpass` auth method allows users to authenticate with Vault using a username and password combination.

Userpasses have to be made manually and can then be linked with other entities, like the one generated by logging in via OIDC Azure AD method.

Enable `userpass` auth method:

Bash

```bash
vault auth enable userpass
```

Then create new user with a random password:

Bash

```bash
vault write auth/userpass/users/<YOUR-USER-HERE> password=<RANDOM-PASS-HERE>
```

And then link it with the users entity identity manually via the GUI from `Access > Entities > USERS-ENTITY > Add alias` with `Name` field value as the same name as entered in the step before and with `Auth Backend` as `userpass/ (userpass)`.

Then, according to `hpc-default` policy, the user can change their password with:

Bash

```
vault write auth/userpass/users/<YOUR-USER-HERE> password="<THEIR-NEW-PASS-HERE>"
```

Client SSH key signing setup

Mount the `ssh-client-signer` secrets engine:

Bash

```bash
vault secrets enable -path=ssh-client-signer ssh
```

Configure Vault with a CA for signing client keys using the `/config/ca` endpoint. If you do not have an internal CA, Vault can generate a keypair for you:

Bash

```bash
vault write ssh-client-signer/config/ca generate_signing_key=true
```

If you already have a keypair, specify the public and private key parts as part of the payload:

Bash

```bash
vault write ssh-client-signer/config/ca \
    private_key="..." \
    public_key="..."
```

Host machine(s) setup

Add the public key to all target host's SSH configuration. This process can be manual or automated using a configuration management tool. The public key is accessible via the API and does not require authentication.

Use:

Bash

```bash
curl -o /etc/ssh/trusted-user-ca-keys.pem $VAULT_ADDR/v1/ssh-client-signer/public_key
```

Or:

Bash

```
vault read -field=public_key ssh-client-signer/config/ca > /etc/ssh/trusted-user-ca-keys.pem
```

Add the path where the public key contents are stored to the SSH configuration file at `/etc/ssh/sshd_config` as the `TrustedUserCAKeys` option:

Text Only

```
TrustedUserCAKeys /etc/ssh/trusted-user-ca-keys.pem
```

Create AuthorizedPrincipalsFile file structure:

Bash

```bash
mkdir /etc/ssh/auth_principals/
```

Create principals with users that can authenticate inside them. With this example can authenticate to `ubuntu@host`:

Bash

```bash
sudo echo "<VAULT-USER-HERE>" > ubuntu
```

Add the path where the principals are stored to the SSH configuration file at `/etc/ssh/sshd_config` as the `AuthorizedPrincipalsFile` option:

Bash

```
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u
```

It is good practice to also disable password auth via SSH at the configuration file at `/etc/ssh/sshd_config`:

Text Only

```
ChallengeResponseAuthentication
noPasswordAuthentication no
```

Restart `sshd` service:

Bash

```bash
sudo systemctl restart sshd
```

Then create separate roles for each user to have granular management over each user. NOTE: setting `algorithm_signer` is especially important (Valid values are `ssh-rsa`, `rsa-sha2-256`, and `rsa-sha2-512`), as `ssh-rsa` is the default but now considered insecure. Note that `allowed_users` controls the principals the Vault user is allowed to sign and connect with. If you need to restrict someone's SSH access, just change their roles `allowed_users` field. Example:

Bash

```bash
vault write ssh-client-signer/roles/my-role -<<"EOH"
{
  "algorithm_signer": "rsa-sha2-512",
  "allow_user_certificates": true,
  "allowed_users": "*",
  "allowed_extensions": "permit-pty,permit-port-forwarding",
  "default_extensions": [
    {
      "permit-pty": ""
    }
  ],
  "key_type": "ca",
  "default_user": "ubuntu",
  "ttl": "8h"
}
EOH
```

It is beneficial for SSH key signing roles use templating to avoid manual policy creations. At the time of writing `hpc-default.hcl` used `ssh-client-signer/sign/{{identity.entity.aliases.<USERPASS-ACCESSOR-ID-HERE>.name}}` for templating. Which means SSH key signing roles must match `userpass` auth method names (which also have to be created manually).

Client side setup

Ask Vault to sign your public key. `valid_principals` field MUST match `ssh-client-signer/sign/<YOUR-ROLE-HERE>` role's `allowed_users`:

Bash

```bash
vault write -field=signed_key ssh-client-signer/sign/sample-role \
 public_key=@$HOME/.ssh/alice-key.pub valid_principals=sample > ~/.ssh/alice-signed-key.pub
```

To customize the signing options, use a JSON payload:

Bash

```bash
vault write ssh-client-signer/sign/my-role -<<"EOH"
{
  "public_key": "ssh-rsa AAA...",
  "valid_principals": "my-user",
  "key_id": "custom-prefix",
  "extensions": {
    "permit-pty": "",
    "permit-port-forwarding": ""
  }
}
EOH
```

(Optional) View enabled extensions, principals, and metadata of the signed key:

Bash

```bash
ssh-keygen -Lf ~/.ssh/signed-cert.pub
```

SSH into the host machine using the signed key. You must supply both the signed public key from Vault and the corresponding private key as authentication to the SSH call:

Bash

```bash
ssh -i signed-cert.pub -i ~/.ssh/id_rsa username@host
```

Host SSH key signing setup (not tested as of writing)

Mount the `ssh-host-signer` secrets engine:

Bash

```bash
vault secrets enable -path=ssh-host-signer ssh
```

Configure Vault with a CA for signing client keys using the `/config/ca` endpoint. If you do not have an internal CA, Vault can generate a keypair for you:

Bash

```bash
vault write ssh-host-signer/config/ca generate_signing_key=true
```

If you already have a keypair, specify the public and private key parts as part of the payload:

Bash

```bash
vault write ssh-host-signer/config/ca \
    private_key="..." \
    public_key="..."
```

Extend host key certificate TTLs:

Bash

```bash
vault secrets tune -max-lease-ttl=87600h ssh-host-signer
```

Create a role for signing host keys. Be sure to fill in the list of allowed domains:

Bash

```bash
vault write ssh-host-signer/roles/hostrole \
    key_type=ca \
    ttl=87600h \
    allow_host_certificates=true \
    allowed_domains="localdomain,example.com" \
    allow_subdomains=true
```

Sign the host's SSH public key:

Bash

```bash
vault write ssh-host-signer/sign/hostrole \
    cert_type=host \
    public_key=@/etc/ssh/ssh_host_rsa_key.pub
```

Set the resulting signed certificate as `HostCertificate` in the SSH configuration on the host machine:

Bash

```bash
vault write -field=signed_key ssh-host-signer/sign/hostrole \
    cert_type=host \
    public_key=@/etc/ssh/ssh_host_rsa_key.pub > /etc/ssh/ssh_host_rsa_key-cert.pub
```

Set permissions on the certificate to be `0640`:

Bash

```bash
chmod 0640 /etc/ssh/ssh_host_rsa_key-cert.pub
```

Add host key and host certificate to the SSH configuration file at `/etc/ssh/sshd_config`:

Text Only

```
# For host keys
HostKey /etc/ssh/ssh_host_rsa_key
HostCertificate /etc/ssh/ssh_host_rsa_key-cert.pub
```

Restart `sshd` service:

Bash

```bash
sudo systemctl restart sshd
```

Client-Side Host Verification

Retrieve the host signing CA public key to validate the host signature of target machines.

Use:

Bash

```bash
curl $VAULT_ADDR/v1/ssh-host-signer/public_key
```

Or:

Bash

```bash
vault read -field=public_key ssh-host-signer/config/ca
```

Backup setup

This assumes that Raft is used as the storage back-end.

Create a backup:

- Manually:

Bash

```
vault operator raft snapshot save demo.snapshot
```

- Automatically with schedule via CLI:

Bash

```bash
vault write sys/storage/raft/snapshot-auto/config/daily interval="24h" retain=5 path_prefix="raft-backup" storage_type="local" local_max_space=1073741824
```

Automatically with schedule with:

Bash

```bash
vault read sys/storage/raft/snapshot-auto/config/daily
```

Restore with:

Bash

```bash
vault operator raft snapshot restore demo.snapshot
```

Other secret engines and use cases

Vault provides more secret engines and use cases that may be of use but were not covered in this test/demo.

Using Key/Value engine as a way to track and renew SSL certs

Certificates get issued after Let's Encrypt validates that users control the domain names in those certificates using the ACME API and "challenges". The most popular ones are the HTTP-01 and the DNS-01. The first requires users to get a particular file and serve it via HTTP or HTTPS, so that the Let's Encrypt servers are able to retrieve it. The latter uses DNS records respectively, so that Let's Encrypt can validate the domain ownership via queries. There are already many clients which ease both of those processes with EFF's Certbot being the most prominent one.

Certbot supports certificate creation and renewal using both challenge types. For dealing with multiple domain names from one server, HTTP-01 challenges seem to be cumbersome: Certbot must serve some traffic on ports 80 and 443 for the Let's Encrypt servers to validate the domains. DNS-01 challenges are better on this perspective, but still this is not to be considered cloud-ready for two reasons:

- The certificate state is stored locally on the server
- The renewal process depends on a running cronjob of the same server

To tackle with the first point, could still use Certbot, but store certificates, tokens etc. in Vault.

Vault engines not covered in test

Database engine

The database secrets engine generates database credentials dynamically based on configured roles. It works with a number of different databases through a plugin interface. There are a number of built-in database types, and an exposed framework for running custom database types for extendability. This means that **services that need to access a database no longer need to hardcode credentials**: they can request them from Vault, and use Vault's leasing mechanism to more easily roll keys. These are referred to as "dynamic roles" or "dynamic secrets".

Since every service is accessing the database with unique credentials, it makes auditing much easier when questionable data access is discovered. You can track it down to the specific instance of a service based on the SQL username.

PKI Secrets Engine

The PKI secrets engine generates dynamic X.509 certificates. With this secrets engine, services can get certificates without going through the usual manual process of generating a private key and CSR, submitting to a CA, and waiting for a verification and signing process to complete. Vault's built-in authentication and authorization mechanisms provide the verification functionality.

By keeping TTLs relatively short, revocations are less likely to be needed, keeping CRLs short and helping the secrets engine scale to large workloads. This in turn allows each instance of a running application to have a unique certificate, eliminating sharing and the accompanying pain of revocation and rollover.

In addition, by allowing revocation to mostly be forgone, this secrets engine allows for ephemeral certificates. Certificates can be fetched and stored in memory upon application startup and discarded upon shutdown, without ever being written to disk.

Transit Secrets Engine

The transit secrets engine handles cryptographic functions on data in-transit. Vault doesn't store the data sent to the secrets engine. It can also be viewed as "cryptography as a

service" or "encryption as a service". The transit secrets engine can also sign and verify data; generate hashes and HMACs of data; and act as a source of random bytes.

The primary use case for transit is to encrypt data from applications while still storing that encrypted data in some primary data store. This relieves the burden of proper encryption/decryption from application developers and pushes the burden onto the operators of Vault.

Key derivation is supported, which allows the same key to be used for multiple purposes by deriving a new key based on a user-supplied context value. In this mode, convergent encryption can optionally be supported, which allows the same input values to produce the same ciphertext.

Datakey generation allows processes to request a high-entropy key of a given bit length be returned to them, encrypted with the named key. Normally this will also return the key in plaintext to allow for immediate use, but this can be disabled to accommodate auditing requirements.

Security/Threat model

Some risks:

- Vault traffic might be eavesdropped
- Solution: Disable unencrypted HTTP comms, only use TLS encrypted traffic.
- User with signed SSH key might insert their own key to authorized keys list.
- Solution: Restrict machine user permissions.
- Solution: Schedule automatic SSH configuration deployment that clears dirs and inserts up-to-date configs to machines.
- Signed SSH key most likely TTL left after revoking SSH signing permissions.
- Solution: Admin generates new CA key.
- Authenticated user might have TTL left after revoking auth permissions.
- Solution: Disable/delete auth entity.